

Sistema de Gestión de Proyectos — Ejercicio 15

JUAN ALBERTO TRUJILLO HERRERA - MARTÍN CAMILO CORREDOR PESCADOR

Universidad Libre Seccional Pereira · Facultad de Ingeniería
Programación II — Cliente/Servidor · Semestre 2026-1

Stack Tecnológico: React + Vite + TailwindCSS | Node.js + Express | MySQL | JWT + bcrypt



Arquitectura del Sistema

Frontend (React/Vite)	Backend (Node/Express)	Base de Datos (MySQL)
http://localhost:5173	http://localhost:5000	localhost:3306



Credenciales de Prueba

Usuario	Contraseña	Rol
admin	admin123	admin
betolio	123456	usuario



Explicación del Modelo Relacional

1. Clientes a Proyectos (1:N):

Un cliente puede tener múltiples proyectos asociados, pero cada proyecto pertenece a un único cliente. La relación se establece mediante el campo `id_cliente` en la tabla *proyectos*, que actúa como llave foránea referenciando a *clientes*.

2. Proyectos a Colaboradores (N:M a través de Participaciones):

Se utiliza la tabla intermedia *participaciones* para manejar la relación muchos a muchos. Un proyecto cuenta con varios colaboradores y un colaborador puede participar en distintos proyectos. Se vinculan mediante `id_proyecto` y `nif_colaborador`.

3. Colaboradores a Pagos (1:N):

Un colaborador puede recibir múltiples pagos a lo largo del tiempo. La tabla *pagos* almacena el `nif_colaborador` para identificar a quién se le realiza el desembolso.

4. Tipos de Pago a Pagos (1:N):

Cada registro de pago debe estar categorizado bajo un tipo específico (ej. transferencia, efectivo, bonificación). La tabla *tipos_pago* define estas categorías y se relaciona con *pagos* mediante `id_tipo_pago`.

5. Usuarios:

Tabla independiente encargada de la autenticación y control de acceso (RBAC). Almacena las credenciales y el `rol` (admin/usuario) para proteger los endpoints de la API.



Endpoints de la API

Método	URL	Descripción	Rol mínimo
POST	/api/auth/login	Iniciar sesión	—
POST	/api/auth/register	Crear usuario	—
GET	/api/dashboard	Estadísticas generales	usuario
GET	/api/clientes	Listar clientes	usuario
POST	/api/clientes	Crear cliente	admin
PUT	/api/clientes/:id	Actualizar cliente	admin
DELETE	/api/clientes/:id	Eliminar cliente	admin
GET	/api/colaboradores	Listar colaboradores	usuario
POST	/api/colaboradores	Crear colaborador	admin
PUT	/api/colaboradores/:id	Actualizar colaborador	admin
DELETE	/api/colaboradores/:id	Eliminar colaborador	admin
GET	/api/proyectos	Listar proyectos	usuario
POST	/api/proyectos	Crear proyecto	admin
PUT	/api/proyectos/:id	Actualizar proyecto	admin
DELETE	/api/proyectos/:id	Eliminar proyecto	admin
GET	/api/pagos	Listar pagos	usuario
POST	/api/pagos	Crear pago	admin
PUT	/api/pagos/:id	Actualizar pago	admin
DELETE	/api/pagos/:id	Eliminar pago	admin
GET	/api/tipos-pago	Listar tipos de pago	usuario
POST	/api/tipos-pago	Crear tipo de pago	admin
PUT	/api/tipos-pago/:id	Actualizar tipo de pago	admin
DELETE	/api/tipos-pago/:id	Eliminar tipo de pago	admin

Estructura del Proyecto

```
gestion-proyectos/
├── frontend/
│   └── src/
│       ├── components/      # Layout y componentes reutilizables
│       ├── context/
│       ├── hooks/
│       ├── pages/           # Vistas (Auth, Dashboard, Clientes, etc.)
│       └── services/        # Consumo de APIs (Axios)
├── backend/
│   ├── config/              # Variables de entorno y ajustes globales
│   ├── controllers/         # Lógica de las rutas (manejadores)
│   ├── database/            # Conexión y configuración de la base de datos
│   ├── middleware/          # Validaciones y seguridad (Auth, JWT)
│   ├── models/              # Definición de estructuras de datos
│   ├── routes/              # Definición de los endpoints de la API
│   └── .env                 # Almacenar variables de entorno
├── database/
│   ├── schema.sql           # Definición de tablas y relaciones
│   └── seeds.sql            # Scripts de datos iniciales
└── README.md
```